CE114: rational d for $36n^3 - 19 = -2d\,x^2\big(-(x+6n) + \sqrt{\cdots}\big)$ via the Booker–Sutherland gap search

rationale and modus operandi of the current scripts

Jamal Agbanwa

12 November 2026; revised 13 November 2026

Abstract

The scripts in `114 /Algorithm for computation /gap-search/` look for rational values of d for which the equation $36n^3-19 = -2d\,x^2\big(-(x+6n)+\sqrt{(x+6n)^2+(36n^3-19)/x}\big)$ has a point with an integral square root (integer chart) or with $x = X/2$ and square root $Y/2$, X, Y odd (half chart). Both acceptable results are exactly the integer solutions of $a^3+b^3+c^3 = 114$: every solution gives one rational d of the first kind and two of the second.

Following Dr. Grechuk's advice to use the best available algorithm and to aim it beyond the range already checked, the production computation is the Booker–Sutherland method. It is restricted to the part of the search space that their 2020 search for $k = 114$ left out: $\delta = |a+b| > 10^{19}/54$, with $3.847\,\delta \le |c| \le 64\,\delta$. A converter then turns any solution found into its three values of d, verified on the original equation. The campaign is sized for 1,800 core-hours.

This note gives the reduction with proofs, what is already excluded, the heuristic that selects the region, the algorithm step by step, the operating procedure, the validation and review evidence, the status of the folder's earlier programs, and the limits of the method. Under the Booker–Sutherland heuristic, the run has about a 1 in 150,000–170,000 chance of producing a solution. (It was 1 in 190,000–200,000 before the 2-adic rule of Proposition 13, which makes the engine about 1.26 times faster.)

Contents

1 Purpose and target **2**

2 The problem is exactly $114 = a^3+b^3+c^3$ **2**

3 What has already been excluded **4**

4 Rationale: where to spend 1,800 core-hours **4**
4.1 Dr. Grechuk's critique of the earlier d-first scripts 4
4.2 The density model 5
4.3 Cost model and choice of region 5
4.4 Calibration and expected yield 5

5 The engine: modus operandi **6**
5.1 Sign and residue rules 6
5.2 Steps for each δ 8

6 From a solution to the rational d: `to_d.py` **9**

7 Operating the 1,800-core-hour campaign **9**

8 Validation and review **10**
8.1 Tests (`make test`) 10
8.2 Pilot 10
8.3 Independent reviews 10

9 Status of the folder's earlier programs **11**

10 What a run proves, and its limits **11**

A Files **12**

1 Purpose and target

Let

$$36n^3 - 19 = -2d\,x^2\big(-(x+6n)+s\big), \qquad s = \sqrt{(x+6n)^2 + \frac{36n^3-19}{x}} \ge 0. \tag{E}$$

The task is to find rational d for which (E) has a point of one of the following kinds:

Result 1. *Integer chart:* $n, x \in \mathbb{Z}$ and $s \in \mathbb{Z}$.

Result 2. *Half chart:* $n \in \mathbb{Z}$, $x = X/2$ and $s = Y/2$ with X, Y odd integers.

The code lives in `https://github.com/JAgbanwa/SomethingNew/tree/main/114%20/Algorithm%20for%20computation%20/gap-search`. This note describes the revision of 6 November 2026 (commits `d19a6a3` and `194e6ca`). The revision of 13 November 2026 adds Remark 7 (this d is not Booker–Sutherland's δ), Proposition 6 (Results 1 and 2 are the only targets, and the only half-integral points that are representations of 114), the new cross-check `rational-d-search/` (Section 9), and the local conditions $n \equiv 0 \pmod 3$, $x \equiv 7 \pmod{12}$ of Result 1 (Proposition 12). The 2-adic part of the latter gives a new rule, Proposition 13. The engine now skips three quarters of the even δ before any root finding, which is about 1.26 times faster on the same host. The search is still $k = 114$ only, and the region is unchanged.

2 The problem is exactly $114 = a^3 + b^3 + c^3$

Throughout, write $N(n) = 36n^3 - 19$ and $U = x + 6n$, and more generally $36n^3 - m$ with $k = 6m$. Production uses $m = 19$, $k = 114$.

Proposition 1 (the cube identity)**.** *For $x \neq 0$ put $A = -x + s$, $B = -x - s$, $C = 2x + 6n$, where s is any number. Then*

$$A^3 + B^3 + C^3 - 6m \;=\; -6x\Big(s^2 - (x+6n)^2 - \frac{36n^3 - m}{x}\Big).$$

Proof. $A^3 + B^3 = (A+B)\big((A+B)^2 - 3AB\big) = -2x(4x^2 - 3x^2 + 3s^2) = -2x^3 - 6xs^2$. Also $C^3 = 8x^3 + 72x^2 n + 216xn^2 + 216n^3$. Adding,

$$A^3 + B^3 + C^3 - 6m = 6x^3 + 72x^2 n + 216xn^2 + 216n^3 - 6m - 6xs^2.$$

The right-hand side of the claim expands to $-6xs^2 + 6x(x^2 + 12xn + 36n^2) + 6(36n^3 - m)$, which is the same. □

Proposition 2 (d is determined by the point)**.** *At a point of* (E)*, $s \neq U$ and*

$$d = -\frac{x + s + 6n}{2x} = \frac{A + C}{A + B}.$$

Proof. $N(n) \neq 0$ for integer n, because 114 is not a cube. So (E) forces $s \neq U$ and $d \neq 0$. From (E), $s-U = -N/(2dx^2)$, and from the definition of s, $(s-U)(s+U) = s^2-U^2 = N/x$. Dividing gives $s+U = -2dx$. Finally $A+C = x+s+6n$ and $A+B = -2x$. □

Proposition 3 (both charts are representations of 114)**.** *A point of Result 1 or Result 2 gives integers A, B, C with $A^3+B^3+C^3 = 114$ and $A+B+C = 6n$. Conversely, let $114 = a^3+b^3+c^3$ with $a, b, c \in \mathbb{Z}$. Choose any one component as C and order the other two as $A \geq B$. Then*

$$n = \tfrac{A+B+C}{6} \in \mathbb{Z}, \qquad x = -\tfrac{A+B}{2}, \qquad s = \tfrac{A-B}{2} \geq 0, \qquad d = \tfrac{A+C}{A+B}$$

is a point of (E)*. It lies in the integer chart if $A+B$ is even and in the half chart (with $X = -(A+B)$ and $Y = A-B$ odd) if $A+B$ is odd.*

Proof. Result 1 gives integers directly. In Result 2, $A = (Y-X)/2$, $B = -(X+Y)/2$, $C = X+6n$ are integers because X, Y are odd. Proposition 1, together with the definition of s, gives the sum of cubes.

For the converse: $114 \equiv 6 \pmod 9$, and cubes are $0, \pm 1 \pmod 9$, so $a \equiv b \equiv c \equiv 2 \pmod 3$ and $3 \mid a+b+c$. Also $a+b+c \equiv a^3+b^3+c^3 = 114 \equiv 0 \pmod 2$, so $6 \mid a+b+c$. The pair A, B has $A+B$ and $A-B$ of the same parity, so x and s are both integers or both odd halves. Then Proposition 1 read backwards gives the square-root condition, $s \geq 0$ by the ordering, and Proposition 2 gives (E) with this d. Note that $A+B \neq 0$, since otherwise $C^3 = 114$. □

Proposition 4 (three values of d per representation)**.** *Every representation of* 114 *has exactly one even component, and its three components are distinct. So it yields exactly three distinct points of* (E)*: one of Result 1, with the even component as C, and two of Result 2.*

Proof. $a+b+c$ is even, so 0 or 2 components are odd. Three even components would make the sum of cubes $\equiv 0 \pmod 8$, but $114 \equiv 2 \pmod 8$. So exactly one is even. If C is the even one, $A+B$ is odd + odd, which is even: integer chart. Otherwise $A+B$ is even + odd, which is odd: half chart, and this happens twice.

Two equal components would give $2a^3 + c^3 = 114$. PARI/GP's unconditional Thue solver, `thue(thueinit(x^3+2,1),114)`, returns no solution, so none exists. The three choices of C then give three different pairs (n, x), since $x = -(A+B)/2$ differs. □

Remark 5. For other $k = 6m$ a representation can have a repeated component, for example $8478 = 2\cdot(-21)^3+30^3$. It then yields only two distinct points; the converter handles this case and lists each point once. As an illustration with a known solution, $42 = (-80538738812075974)^3 + 80435758145817515^3 + 12602123297335631^3$ ($m = 7$) gives $n = 2083190438512862$ and

$$d \in \Big\{ -\tfrac{102980666258459}{93037881443153146} \text{ (Result 1)}, \; -\tfrac{93037881443153146}{67936615514740343}, \; -\tfrac{93037881443153146}{102980666258459} \text{ (Result 2)} \Big\}.$$

Proposition 6 (nothing else is targeted)**.** *Let $n \in \mathbb{Z}$, $x \in \frac{1}{2}\mathbb{Z} \setminus \{0\}$ and $s \in \frac{1}{2}\mathbb{Z}$ satisfy the square-root condition $s^2 = (x+6n)^2 + N(n)/x$. Then exactly one of the following holds:*

(i) $x, s \in \mathbb{Z}$: Result 1;

(ii) $x = X/2$, $s = Y/2$ with X, Y odd: Result 2;

(iii) $x \in \mathbb{Z}$ and $s = Y/2$ with Y odd.

The fourth combination, $x = X/2$ with X odd and $s \in \mathbb{Z}$, is impossible. Case (iii) is not a target of this project and is not searched. It corresponds to representations of $8 \cdot 114 = 912$ with two odd components, not to representations of 114*.*

Proof. If $x = X/2$ with X odd, multiplying the square-root condition by $4X$ gives $X\big(4s^2-(X+12n)^2\big) = 8N(n)$. When $s \in \mathbb{Z}$ the left side is odd (it is a product of two odd numbers), while the right side is even, a contradiction. In case (iii), Proposition 1 applied to $2A, 2B, 2C$ (odd, odd, even) gives $(2A)^3+(2B)^3+(2C)^3 = 8\cdot 114$. □

For example, the known point $(n,x) = (-1160307, -10431164)$ has $s = 35094609/2$. It is of type (iii), so it is neither Result 1 nor Result 2, and the campaign does not look for such points.

Remark 7 (this d is not Booker–Sutherland's δ). Booker and Sutherland write $\delta = |a+b|$ for the two components of largest absolute value (Section 3). It is only a search coordinate. In the point of Proposition 3 whose C is the component of smallest absolute value, $\delta = |A+B| = 2|x|$ (Result 1) or $|X|$ (Result 2). The d of (E) is a different quantity, $d = (A+C)/(A+B) = -(s+x+6n)/(2x)$. This expression is d itself, with its sign (for instance $d = -1/54$ at the non-integral point $(1,-9)$), and not $|d|$. The engine walks δ, and `to_d.py` computes the three values of d afterwards.

Consequence. Finding a rational d of either acceptable kind is *the same problem* as finding an integer solution of $a^3+b^3+c^3 = 114$. The best algorithm for the task is therefore the best known algorithm for that sum-of-three-cubes problem, followed by the conversion of Proposition 3.

3 What has already been excluded

Booker and Sutherland [1] parametrize a solution with $|a| > |b| > |c|$ as follows.

Lemma 8 (Booker–Sutherland)**.** *Put* $\delta = |a+b|$ *and* $\alpha = 2^{1/3}-1 \approx 0.259921$. *If* $|c| > \sqrt{|k|}$, *then* $0 < \delta < \alpha|c|$, $c^3 \equiv k \pmod{\delta}$, *and*

$$(a-b)^2 = \frac{4(z^3-K)/\delta - \delta^2}{3}, \qquad z = |c|,\ K = k\ \mathrm{sign}(c).$$

The bound follows from $|a| \ge |b| \ge |c|$, $a \approx -b$ and $\delta(3b^2+3|b|\delta+\delta^2) \approx |c|^3 \le |b|^3$, which gives $(1+\delta/|b|)^3 \le 2$. The formula for $(a-b)^2$ comes from $a^3+b^3 = (a+b)\big((a+b)^2-3ab\big)$.

For $k = 114$ their published searches [1] were:

- 2019: $z_{\max} = 10^{17}$ with $\delta \le \alpha z_{\max}$. This is complete for $|c| \le 10^{17}$.
- 2020 (Charity Engine, together with 390, 579, 627, 633, 732, 921, 975): $z_{\max} = 10^{19}$ with $\delta \le z_{\max}/54 = 185185185185185185$.

No later search has been published. Because $\delta < \alpha|c|$, every δ above $10^{19}/54$ is unsearched for *every* c (Figure 1). A consequence for (E), shown in `HALF_INTEGER.md`, is that any acceptable point needs $|n| \ge 9.8\cdot 10^{15}$ and certificates $V = dX^2$ (resp. $U = 2dx^2$) of at least 25 digits.

4 Rationale: where to spend 1,800 core-hours

4.1 Dr. Grechuk's critique of the earlier d-first scripts

The earlier 390 scripts enumerated rational slopes $d = p/q$ by height. A solution can be reached only if $g = \gcd(A+B, A+C)$ is enormous, because its components have size about $H(d)\cdot g$. The record solutions (for $k = 33, 42, 3, 165, 579, 906$) all have $g = 1$, and the largest value among them is $g = 16$ (for 795). Their heights are $H \sim 10^{15}$–10^{20}. For random pair sums $P(\gcd \ge G) \approx 6/(\pi^2 G)$, so reaching beyond 10^{17} at $H \le 10^6$ has negligible probability. Dr. Grechuk's conclusion, to use the best known method and aim it beyond the checked range, is adopted here.

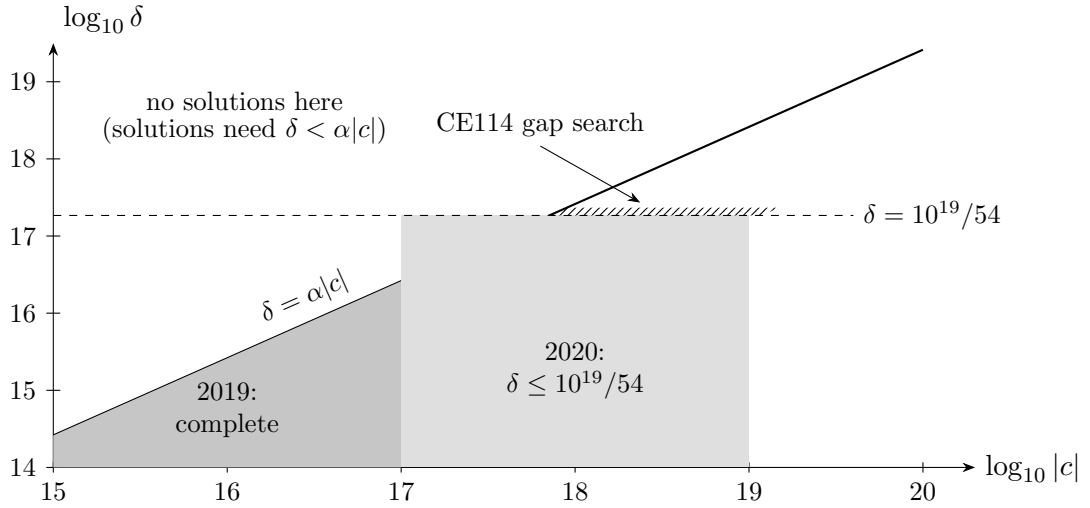


Figure 1: The $(|c|, \delta)$ plane for $k = 114$, log scales. Shaded: searched by Booker–Sutherland. Hatched: the strip searched by this campaign, just above $\delta = 10^{19}/54$, with $3.847\,\delta \le |c| \le 64\,\delta$. The strip's height is exaggerated: 1,800 core-hours cover a relative δ-width of about 1.5×10^{-4}.

4.2 The density model

Booker and Sutherland [1, §2.2] model the share of solutions with a given δ that have $|c|/\delta \ge R$ as $cK(R)$, with $K(R) \sim R^{-1/2}$ and $c \approx 1.96084$, so that $cK(1/\alpha) = 1$. Normalizing by Heath-Brown's conjectured count $e(B) = \rho \log B$ gives a density in the $(\delta, |c|)$ plane of

$$dE = \frac{\rho}{2\sqrt{\alpha}}\,\delta^{-1/2}|c|^{-3/2}\,d\delta\,d|c| \qquad (\alpha|c| > \delta),$$

with $\rho_{114} = 0.058459$ from their Table 1. Two consequences follow.

1. Integrating over $|c| > \delta/\alpha$ gives ρ/δ expected solutions per δ: solutions are *log-uniform in* δ.

2. A band $|c| \le R\delta$ keeps a share $1 - 1/\sqrt{\alpha R}$, so

$$E\big(\delta \in [D_0, D_1),\ |c| \le R\delta\big) = \rho\Big(1 - \frac{1}{\sqrt{\alpha R}}\Big)\ln\frac{D_1}{D_0}. \tag{1}$$

4.3 Cost model and choice of region

In the engine (Section 5) most of the cost is fixed per δ: sieving, early rejection and cube roots. At $R = 64$ the walk over c adds only about 1.5 candidates per δ. The figure that matters is therefore the *yield per* δ, $\rho(1 - 1/\sqrt{\alpha R})/\delta$. It is largest at the smallest unsearched δ, which is $D_0 = 10^{19}/54 + 1 = 185185185185185186$.

The alternatives are worse:

- Extending $|c|$ beyond 10^{19} for $\delta \le 10^{19}/54$ means processing all 1.85×10^{17} smaller δ again before any new c is reached. That is millions of core-hours.
- Small pair sums with huge $|c|$, the direction of the folder's `xroots` and `xcurve`, hold almost nothing beyond 10^{19}. Integrating the density over $\delta \le D$ and $|c| \ge 10^{19}$ gives $2\rho\sqrt{D/(\alpha \cdot 10^{19})}$, about 7×10^{-9} for all $\delta \le 10^4$ combined.

4.4 Calibration and expected yield

Calibration at $\delta \approx 1.85 \times 10^{17}$ for $k = 114$, with the 2-adic rule, on the host of the 13 November revision (one core per measurement, `gap_campaign.py --calibrate`) gave:

R	μs per δ	coverage $1 - 1/\sqrt{\alpha R}$	relative yield per core-second
24	0.237	0.600	2.53×10^6
32	0.246	0.653	2.65×10^6
40	0.253	0.690	2.73×10^6
48	0.250	0.717	2.87×10^6
64	**0.265**	**0.755**	$\mathbf{2.85 \times 10^6}$
80	0.267	0.781	2.93×10^6

The optimum is flat (within 3% from $R = 48$ to 80) and $R = 64$ is kept. With $(R+18)\delta < 2^{64}$ the hard ceiling is $R = 81$ at this δ.

Run back to back on the same host over two windows of 3×10^7 values of δ ($R = 64$), the engine before the 2-adic rule took 0.315–0.323 μs per δ and the current one 0.249–0.260 μs. That is a factor of about 1.26. The original development host measured 0.293 μs for the old engine, which corresponds to about 0.233 μs now. At that speed, 1,800 core-hours cover 2.78×10^{13} values of δ, namely $\delta \in [185185185185185186, 185212996343983469)$. (It was 2.21×10^{13} before.) Equation (1) then gives:

Use of 1,800 core-hours ($k = 114$)	expected solutions
This campaign, 0.233 μs per δ	6.6×10^{-6} (1 in 151,000)
This campaign, 0.253–0.265 μs (this revision's host)	1 in 164,000–172,000
Before the 2-adic rule (0.293 μs)	5.3×10^{-6} (1 in 190,000)
`xroots` (Algorithm D), 1,800 h inside its planned box	$\sim 10^{-11}$
`xcurve` on every $\lvert x\rvert \le 5000$, solved completely	$\sim 7 \times 10^{-9}$ in total
`dsweep`, `dsmooth`, `xscan`	0 (Section 9)

The chance grows linearly with hours. Even so, this code can reach at most about 0.9% (about 2.6×10^6 core-hours at 0.233 μs per δ) before $(R + 18)\delta$ reaches 2^{64} at $R = 64$. These are heuristic figures, resting on Heath-Brown's conjecture and the Booker–Sutherland model, not on theorems.

5 The engine: modus operandi

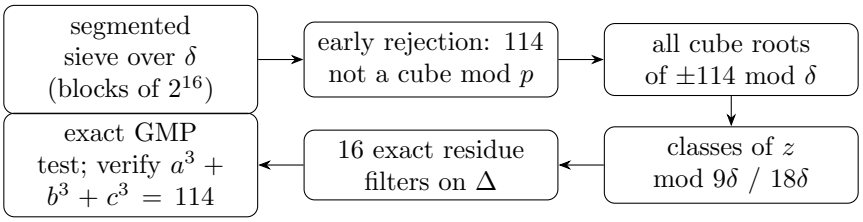


Figure 2: Per-δ pipeline of `src/gap_worker.cpp`. About 85% of δ stop at the second box.

The worker is `src/gap_worker.cpp` (C++17 with the GMP C API). It is invoked as `gap_worker --k 114 --d-start A --d-stop B --ratio-max 64` and processes every $\delta \in [A, B)$. It is the engine written and reviewed for the 390 campaign, with $k = 114$ as its default.

5.1 Sign and residue rules

Proposition 9. *Let $k \equiv 3\varepsilon \pmod 9$, $\varepsilon = \pm 1$; for $k = 114$, $\varepsilon = -1$. Every solution has $a \equiv b \equiv c \equiv \varepsilon \pmod 3$. Hence $3 \nmid \delta$, $z \equiv \delta \pmod 3$, and $\operatorname{sign}(c) = +1$ exactly when $\delta \equiv \varepsilon \pmod 3$. For 114, $c > 0$ exactly when $\delta \equiv 2 \pmod 3$.*

Proof. The first claim is the cube argument mod 9. Then $a+b \equiv 2\varepsilon \equiv -\varepsilon$, so $3 \nmid \delta$. For $|c|$ large, $(a-b)^2 > 0$ forces $\operatorname{sign}(a+b) = -\operatorname{sign}(c)$. If $c > 0$ then $a+b = -\delta \equiv -\varepsilon$, so $\delta \equiv \varepsilon$ and $z = c \equiv \varepsilon$. If $c < 0$ then $\delta = a+b \equiv -\varepsilon$ and $z = -c \equiv -\varepsilon$. In both cases $z \equiv \delta \pmod 3$. This agrees with $s = \varepsilon(\delta/3)$ in [1]. □

Proposition 10 (candidate test and converse)**.** *Put $K = k \operatorname{sign}(c)$ and $\Delta = \big(4(z^3 - K)/\delta - \delta^2\big)/3$. A pair (δ, z) with $z^3 \equiv K \pmod\delta$ comes from a solution if and only if $\Delta = t^2$ for an integer $t \equiv \delta \pmod 2$. The solution is then $s_0 = -\operatorname{sign}(c)\delta$, $a = (s_0 + t)/2$, $b = (s_0 - t)/2$, $c = \operatorname{sign}(c)\, z$.*

Proof. $a^3+b^3 = s_0(s_0^2-3ab) = k-c^3$ gives $ab = (s_0^2-(k-c^3)/s_0)/3$. Then $(a-b)^2 = s_0^2-4ab = \Delta$. Conversely, such a t makes a, b integers with sum s_0 and product ab, and $a^3+b^3+c^3 = k$ follows. The worker re-checks this identity exactly. □

Proposition 11 (classes of z)**.** *$9 \mid \Delta$ for every solution, and $\Delta \bmod 9$ depends only on $z \bmod 9\delta$. For odd δ, $\Delta \equiv 1 \pmod 8$, and $\Delta \bmod 8$ depends only on $z \bmod 2$.*

Proof. $a \equiv b \pmod 3$ gives $9 \mid (a-b)^2$. Write $3\delta\Delta = 4(z^3-K)-\delta^3$. Replacing z by $z+9\delta$ changes z^3 by a multiple of 27δ, so $3\delta\Delta \bmod 27\delta$, and hence $\Delta \bmod 9$, depends only on $z \bmod 9\delta$. For odd δ, t is odd, so $\Delta = t^2 \equiv 1 \pmod 8$. Moreover $3\Delta \equiv 4w - \delta^2 \pmod 8$ with $w \equiv (z^3 - K)\delta^{-1} \equiv z - K \pmod 2$. □

Proposition 12 (local conditions of Result 1)**.** *Every point of Result 1 has $n \equiv 0 \pmod 3$ and $x \equiv 7 \pmod{12}$. Every point of Result 2 has $n \equiv 0 \pmod 3$ and $X \equiv 5$ or $11 \pmod{12}$.*

Proof. By Proposition 3, $A, B, C \equiv 2 \pmod 3$. Write each as $2 + 3u$; then $(2+3u)^3 \equiv 8 + 9u \pmod{27}$, so $114 \equiv 24 + 9(u_A + u_B + u_C) \pmod{27}$, i.e. $u_A + u_B + u_C \equiv 1 \pmod 3$. Hence $A + B + C = 6 + 3\sum u \equiv 0 \pmod 9$. It is also even, so $18 \mid A + B + C = 6n$ and $3 \mid n$. Next, $A + B \equiv 1 \pmod 3$, so $x = -(A+B)/2 \equiv 1 \pmod 3$, and in Result 2 $X = -(A+B) \equiv 2 \pmod 3$ with X odd. In Result 1, A and B are odd and C is even, so $C^3 \equiv 0 \pmod 8$. Odd t satisfy $t^3 \equiv t \pmod 8$, so $A + B \equiv A^3 + B^3 \equiv 114 \equiv 2 \pmod 8$, and $x = -(A+B)/2 \equiv 3 \pmod 4$. Together these give $x \equiv 7 \pmod{12}$. □

In Booker–Sutherland coordinates ($a + b = -\operatorname{sign}(c)\,\delta$), $3 \mid n$ reads $z \equiv \delta \pmod 9$. This is already enforced by the classes of Proposition 11: every one of the 154,376 candidates walked on 10^5 consecutive δ from D_0 satisfied it. The 2-adic half of $x \equiv 7 \pmod{12}$, on the other hand, was not used before. It is the following rule.

Proposition 13 (2-adic rule for even δ)**.** *Let $k \equiv 2 \pmod 4$ (for example $k = 114$) and let δ be even with $\lfloor 3.847\,\delta \rfloor^3 > |k|$. Every solution with this δ in the walk satisfies*

$$-\operatorname{sign}(c)\,\delta \equiv k \pmod 8.$$

For $k = 114$, combined with Proposition 9, the even δ that can occur are $\delta \equiv 10$ or $14 \pmod{24}$: two of the eight even classes prime to 3.

Proof. Since $k \equiv 2 \pmod 4$, exactly one component is even (three even ones would give $8 \mid k$). If $\delta = |a+b|$ is even, a, b have the same parity, so they are the odd pair and c is even. As in the previous proof, $a + b \equiv a^3 + b^3 = k - c^3 \equiv k \pmod 8$. Moreover $a + b$ has the sign of $a^3 + b^3 = k - c^3$, and that sign is $-\operatorname{sign}(c)$ when $|c|^3 > |k|$, which holds on the whole walk $|c| \geq \lfloor 3.847\,\delta \rfloor$. So $a + b = -\operatorname{sign}(c)\,\delta$. For 114: if $c > 0$ then $\delta \equiv 2 \pmod 3$ and $-\delta \equiv 2 \pmod 8$, so $\delta \equiv 14 \pmod{24}$. If $c < 0$ then $\delta \equiv 1 \pmod 3$ and $\delta \equiv 2 \pmod 8$, so $\delta \equiv 10 \pmod{24}$. □

For $k = 114$ this is exactly $x \equiv 3 \pmod 4$ for the Result-1 point whose C is the smallest component. The worker marks the excluded δ when it sets up each sieve block, so it neither factors them nor looks for cube roots. It counts them as `d_skipped_2adic`. `--no-filter` switches the rule off, for validation.

5.2 Steps for each δ

1. **Factor.** Primes up to $\sqrt{\delta_{\text{stop}}} \approx 4.3 \times 10^8$ are generated once per task in about 1–3 s. Primes below 2^{16} are applied directly to each block of 2^{16} consecutive δ. Larger primes live in buckets: linked lists of fixed-size pages drawn from a shared free pool, each prime filed under the block of its next multiple. This keeps peak memory at about 0.24 GB, measured across more than two full bucket cycles. The cofactor left after sieving is 1 or a prime.

2. **Reject early.** A prime $p \equiv 1 \pmod 3$ dividing δ for which 114 is not a cube mod p leaves no root. This is precomputed per prime, packed into the bucket entries, and applied before any division. Similarly $p \mid 114$ with $v_p(114) = 1$ and $p^2 \mid \delta$ leaves no root. One third of all δ are skipped because $3 \mid \delta$, and a quarter by the 2-adic rule (Proposition 13). Of the rest, about 29% of all δ have no root, so only about 12% remain.

3. **Cube roots of K mod δ.**

   - $p \equiv 2 \pmod 3$: $K^{(2p-1)/3}$.
   - $p \equiv 1 \pmod 3$: Adleman–Manders–Miller in the 3-Sylow subgroup, with a single-exponentiation fast path when $p \equiv 4, 7 \pmod 9$.
   - Then Hensel lifting to p^e, brute force for $p \mid 114$, and CRT across the factors.

   Roots of -114 are the negatives of roots of 114. Exponentiations use 64-bit Montgomery arithmetic.

4. **Walk.** For each root, keep the classes modulo 9δ (or 18δ for odd δ) allowed by Propositions 9 and 11. Walk them over $\lfloor 3847\delta/1000 \rfloor \le z \le R\delta$. Since $3.847 < 1/\alpha = 3.84732\ldots$, no admissible z is skipped; the k-correction to Lemma 8 is about 10^{-30} relative.

5. **Filter.** For each of the 16 primes $5 \le p \le 61$ not dividing δ, test that $\Delta \equiv A_p z^3 + B_p \pmod p$ is a square (zero included), with $A_p = 4/(3\delta)$ and $B_p = -4K/(3\delta) - \delta^2/3$. Each test is a necessary condition, compiled with a constant modulus.

6. **Decide and verify.** For survivors, compute Δ exactly with GMP, test whether it is a perfect square with $t \equiv \delta \pmod 2$, rebuild (a, b, c) by Proposition 10, and verify $a^3 + b^3 + c^3 = 114$. The hit is printed as a JSON line.

Measured on 16 million δ starting at D_0 ($k = 114$, $R = 64$, with the 2-adic rule):

counter	total	per δ
δ skipped, $3 \mid \delta$	5,333,333	0.333
δ skipped by the 2-adic rule	3,999,999	0.250
δ with no cube root	4,679,745	0.292
δ processed further	1,986,923	0.124
cube roots	4,618,747	0.289
candidates z	18,524,758	1.16
passed the residue filters (exact GMP checks)	647	4.0×10^{-5}
solutions	0	–

Overflow guards. The worker refuses to start unless $27\delta_{\text{stop}} < 2^{63}$ and $(R + 18)\delta_{\text{stop}} < 2^{64}$. The 2-adic residue of Δ is computed exactly mod 2^{64} using δ^{-1} and 3^{-1}. All other modular products stay within 128 bits.

6 From a solution to the rational d: `to_d.py`

`to_d.py` reads `hits.jsonl` (or raw worker output, skipping non-hit records), or a single triple via `--triple A B C`. It removes duplicate representations and, for each choice of C, applies Proposition 3. Before printing, it checks every point with exact rationals:

- $s \geq 0$ and $s^2 = (x + 6n)^2 + N/x$;
- equation (E) holds with this d;
- the chart: $x, s \in \mathbb{Z}$ (Result 1), or both with denominator 2, so that $X = 2x$ and $Y = 2s$ are odd (Result 2).

Output, one line per point:

```
result 2 (half chart):  d = ...  n = ...  x = .../2 sqrt = .../2 X=...  Y=...
```

By Proposition 4, a representation of 114 always produces exactly one Result-1 line and two Result-2 lines.

7 Operating the 1,800-core-hour campaign

1. **Build and test:** `make && make test` (19 tests, about 140 s). Or `docker build .`, which compiles the worker and runs the test suite during the build.
2. **Calibrate** on Charity Engine's CPU type: `python3 gap_campaign.py --calibrate`. It prints the μs per δ for several R and the best R.
3. **Plan:** `python3 gap_campaign.py --campaign-id ce114-gap --ratio-max 64 --us-per-d <measured> --output-dir gap-tasks`. The defaults are $k = 114$, 1,800 core-hours and $D_0 = 185185185185185186$. The range is split evenly into tasks of about one hour each (1,800 tasks; always pass the μs per δ measured on the grid), and `page.json` is written with the heuristic yield. Nothing is submitted.
4. **Run** each task: `python3 run_gap_task.py --task <task.json> --output-dir <dir>`. In the container this is the default command, on `/local/input/task.json`.
   - The runner verifies each hit exactly before storing it.
   - It writes an fsynced checkpoint every 2^{24} values of δ, recording the exact interval [covered_start, d_next) this directory has searched.
   - It resumes in place. It refuses to resume if the task would start before that interval, so no unsearched range is ever credited.
   - On SIGTERM it stops the worker and writes `continuation.task.json`.
   - Exit code: 0 for complete or partial; `--partial-exit-code` can make partial runs non-zero so the grid retries them.
5. **Collect:** `collect_gap.py --page ... --results ... --output-dir ...` unions the checkpoint intervals and validates them against tasks and page. It reports gaps (exit 2 with `--require-complete`), re-verifies every hit, and sums worker core-hours.
6. **Audit:** `audit_gap.py --fraction 0.02` re-runs a random 2% of returned chunks on trusted hardware and compares their deterministic counters. Any mismatch flags a faulty host; that task is re-issued.
7. **Convert:** `python3 to_d.py gap-collected/hits.jsonl`.

Resources. About 0.24 GB of memory, 1–3 s of set-up per task, single-threaded. Set-up and the 2% audit (about 36 core-hours) come on top of the 1,800 hours and should be kept within the project's headroom.

8 Validation and review

8.1 Tests (`make test`)

- **Record solutions.** From ± 25-wide δ windows the worker rediscovers the known solutions for $k = 33, 42, 3, 165, 579, 795, 906$. These include 579, found in the same 2020 run that covered 114, and $k = 3$ at $|c|/\delta \approx 4.4 \times 10^6$. Excluding the window loses the hit, and disabling every filter changes nothing.
- **Brute force.** For every $k \equiv \pm 3 \pmod 9$ up to 100, the worker finds all solutions with coordinates up to 2000 in the searched region, and nothing else.
- **2-adic rule.** The current engine and the engine before the rule return identical hits for all 55 values $k < 1000$ with $k \equiv 2 \pmod 4$ and $k \equiv \pm 3 \pmod 9$, over $1 \le \delta < 3 \times 10^6$ with $|c| \le 1000\,\delta$ (143 representations). The test suite repeats this against `--no-filter` for $k = 6, 42, 222, 438, 618$. It also checks the congruence on every even-δ hit, and that the skip was active.
- **Filter decisions.** On production-size candidates for $k = 114, 390, 627, 33$, every 2-adic and residue decision agrees with exact integer arithmetic.
- **Factorizations and roots.** Factorizations near D_0 and near 2^{58} are checked by product and primality. Root sets are checked by brute force for $\delta < 1500$, and by multiplicativity at production size for $k = 114$ and 390, including δ divisible by 2^{25}, 5^{14}, 7^{12} and 13^{10}.
- **Converter.**
  - The 42 and 906 records give $1 + 2$ points satisfying (E), with the same d as the folder's independent `grid/verify.py`.
  - A brute force over (E) itself ($|n| \le 25$, $0 < |X| \le 120$, nine values of m including 19) confirms that every acceptable point is a representation and is listed. For $m = 19$ there are none, as Booker–Sutherland's bound implies.
  - Raw worker output and duplicated hits are handled; a repeated component gives two points.
- **Lifecycle.** Plan, run, SIGTERM, resume and collect work end to end. Re-running an original task where its continuation ran is refused, a same-directory continuation leaves no false gap, and the audit detects a counter altered by one.

8.2 Pilot

A pilot over $\delta \in [D_0, D_0 + 2 \times 10^9)$ ($R = 64$) ran through the full pipeline. It used 0.173 core-hours (0.31 μs per δ on a shared host), returned no solutions, left no gaps, and 6 of 6 audited chunks agreed. The record is in `gap-search/validation/pilot/`.

8.3 Independent reviews

Two separate reviews read the code without being given the author's conclusions.

- **The engine.** The first review re-checked roots, factorizations, the z-walk and filter decisions on several million further δ and 10^5 further candidates, and found no way to miss a solution. Its findings on coverage accounting, durability, the mod-9δ speed-up and the figures were all adopted.
- **The 114 parts.** The second review covered the converter, the equivalence claims, the 114 port and the documentation. It confirmed the identity, the congruences, the yield arithmetic and the search boundary. Its corrections (the description of the older engines, distinctness of components, reach of the code, CLI details, even task split) were applied in commit `194e6ca`.

9 Status of the folder's earlier programs

The earlier programs in `grid/` and `search/` remain for reproducibility. In the light of Section 3 they are not the place where an acceptable result can come from:

program	status for Results 1 and 2
`dsweep`, `dsmooth`	They enumerate the certificate $U = 2dx^2$ (or $V = dX^2$) and do have integral modes. But an acceptable point needs $\|V\| \geq 5.8 \cdot 10^{24}$; `dsweep` stops at 10^{18}, and `dsmooth` reaches that size only for B-smooth certificates.
`xscan`	Scans n-windows far below the required $\|n\| \geq 9.8 \cdot 10^{15}$.
`xcurve`	Unbounded in n for each fixed x, but small $\|x\|$ (small pair sums) holds about 7×10^{-9} expected solutions beyond $\|c\| = 10^{19}$ for all $\|x\| \leq 5000$.
`xroots` (Algorithm D)	Correct and complete for its boxes, but aimed at small $\|X\| < 10^6$ with $\|n\|$ beyond 10^{18}: about 10^{-11} per 1,800 hours.
`rational-d-search/` (new)	Every (n, x) in both charts whose square root is merely *rational*, complete in x for each n. It factors $36n^3 - 19$ completely (rho and ECM), matches a brute force for $\|n\| \leq 450$, and re-finds the eleven known integer-chart points. It is a cross-check only: its points are not Results 1 or 2.
`RequestProject/*.lean`	Lean proofs of the reduction, the small-$\|n\|$ completeness and the earlier algorithms; unaffected.

The rational-d points with non-integral radicals found by those programs (for example $d = -142/4913$ at $(n, x) = (15, -289/2)$, or the new $d = -41449/33275$ at $(309, 15125/2)$ from `rational-d-search`) are genuine points of (E). They are not Results 1 or 2.

10 What a run proves, and its limits

A completed and audited campaign proves that (E) has *no* point of Result 1 or 2 arising from a representation with $\delta = |a + b|$ in the covered range and $3.847\,\delta \leq |c| \leq 64\,\delta$. Equivalently, it proves that 114 has no representation in that region. This is a new, exact exclusion region that does not depend on any heuristic.

It does not cover other δ, or $|c|$ outside the band. The probability figures of Section 4 are heuristic. Three things would raise the odds:

- more core-hours, since the gain is linear;
- Booker and Sutherland's own implementation, which a rough comparison with their $k = 33$ benchmark suggests may be about three times faster per δ;

- confirming with them, or with Charity Engine, that the region has not been searched since 2020.

A Files

file (in `gap-search/`)	role
`src/gap_worker.cpp`	the engine (Section 5)
`gap_common.py`	shared validation, exact hit verification, density table, yield formula
`gap_campaign.py`	calibration, budget sizing, task and `page.json` generation
`run_gap_task.py`	restartable task runner with checkpoints and continuation
`collect_gap.py`	coverage union, gap report, hit re-verification
`audit_gap.py`	re-runs sampled chunks to detect faulty hosts
`to_d.py`	representation $\to$ the three rational d, verified on (E)
`tests/`	`test_gap.py` (engine, 13 tests), `test_to_d.py` (converter, 6 tests)
`Dockerfile`, `container_entrypoint.sh`	Charity Engine image (tests run at build)
`examples/pilot/`, `validation/pilot/`	pilot tasks and its collected summary
`README.md`	short operating guide

References

[1] A. R. Booker and A. V. Sutherland, *On a question of Mordell*, Proc. Natl. Acad. Sci. USA **118** (2021), e2022377118; arXiv:2007.01209.

[2] D. R. Heath-Brown, *The density of zeros of forms for which weak approximation fails*, Math. Comp. **59** (1992), 613–623.